Streaming Stochastic Submodular Maximization with On-Demand User Requests

Honglian Wang
KTH Royal Institute of Technology
Stockholm, Sweden

Sijing Tu
KTH Royal Institute of Technology
Stockholm, Sweden

Lutz Oetterhagen
University of Liverpool
Liverpool, UK

Aristides Gionis
KTH Royal Institute of Technology
Stockholm, Sweden

Abstract

We explore a novel problem in streaming submodular maximization, inspired by the dynamics of news-recommendation platforms. We consider a setting where users can visit a news website at any time, and upon each visit, the website must display up to k news items. User interactions are inherently stochastic: each news item presented to the user is consumed with a certain acceptance probability by the user, and each news item covers certain topics. Our goal is to design a streaming algorithm that maximizes the expected total topic coverage. To address this problem, we establish a connection to submodular maximization subject to a matroid constraint. We show that we can effectively adapt previous methods to address our problem when the number of user visits is known in advance or linear-size memory in the stream length is available. However, in more realistic scenarios where only an upper bound on the visits and sublinear memory is available, the algorithms fail to guarantee any bounded performance. To overcome these limitations, we introduce a new online streaming algorithm that achieves a competitive ratio of $1/(8\delta)$, where δ controls the approximation quality. Moreover, it requires only a single pass over the stream, and uses memory independent of the stream length. Empirically, our algorithms consistently outperform the baselines.

1 Introduction

Streaming submodular maximization has been extensively studied under various feasibility constraints, including *matchoid* [7, 8, 10], *matroid* [9], *knapsack* [21, 22] and k-*set* [20] constraints. In the classical setting, the goal is to select a single subset of items from a data stream and output it when the stream ends. However, this classical framework falls short in many real-world scenarios, for example, when seeking to maximize *content coverage* in online news recommendation. In such a setting, users visit a news website multiple times and expect to receive a relevant set of articles on each visit. Meanwhile, new articles continuously arrive at the content-provider's server in a stream fashion. The long-term objective is to maximize the total content coverage across user visits. Existing streaming submodular maximization algorithms are ill-suited for this use case, as they are designed to produce a single solution and do not support multiple or on-demand user interactions during the stream.

Motivated by this limitation, we introduce a novel problem called *Streaming Stochastic Submodular Maximization with On-demand Requests* (*S3MOR*). While the setting is broadly applicable to streaming submodular maximization tasks with multiple output requirements, we focus on the news recommendation task for illustration. Therefore, in *S3MOR*, the data stream consists of incoming

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

Table 1: Comparison of our algorithms. $N \in \mathbb{N}$ denotes the stream length, $k \in \mathbb{N}$ number of news items to present per user access, $T \in \mathbb{N}$ and $T' \in \mathbb{N}$ are the exact number and upper bound of user accesses, and $\delta \in \mathbb{N}$ an approximation parameter.

Algorithm	**Competitive ratio**	**Time**	**Space**	**Response time**
LMGREEDY	$1/2$	$\mathcal{O}(NT'k)$	$\Theta(N + kT)$	$\mathcal{O}(Nk)$
STORM	$1/4(T'-T+1)$	$\mathcal{O}(NT'k)$	$\mathcal{O}(T'k)$	$\mathcal{O}(T')$
STORM++	$1/(8\delta)$	$\mathcal{O}(NT'^2k/\delta)$	$\mathcal{O}(T'^2k/\delta)$	$\mathcal{O}(T')$

news items that the system aggregates into a personalized news website. Each item V_i covers a set of topics and is associated with a probability p_i, representing the probability that a user will click on the item when it is presented. A user may visit the system arbitrarily often and at any time during the stream. At each visit, the system must select and present a subset of at most k items from those that have arrived in the stream up to that point. If a user clicks on item V_i, they are exposed to all topics associated with that item. The objective is to maximize the expected total number of unique topics covered by all user clicks across all visits. This setting is challenging due to its online character:

- **On-demand**: At any point, the system must be ready to output a subset of news items.
- **Irrevocability**: Once a subset is presented to the user, it cannot be modified.

To the best of our knowledge, no existing algorithm directly addresses the *S3MOR* problem.

We evaluate the performance of online streaming algorithms that make irrevocable decisions using the notion of *competitive ratio*. An algorithm is called *c-competitive* if, for every input stream σ, the value $\mathcal{A}(\sigma)$ achieved by algorithm $\mathcal{A}$ satisfies $\frac{\mathcal{A}(\sigma)}{\mathbf{OPT}(\sigma)} \geq c$, where $\mathbf{OPT}(\sigma)$ denotes the value of an optimal offline solution with full access to the input stream.

To evaluate the performance of our algorithms, we clarify the notions of *response time complexity*, *time complexity*, and *space complexity*.

Response-time complexity vs. time complexity. For the *S3MOR* problem, we assume there exists an oracle that can evaluate the objective function $f(S)$ for any feasible candidate set S. We define the *response time complexity* of an algorithm as the worst-case number of oracle calls needed to produce a recommendation result of size k after the user submits a request. This metric captures the latency users experience. In contrast, *time complexity* is defined as the worst-case number of oracle calls required by the algorithm over the entire stream.

Space complexity. In practice, all users share common storage for original news items. We distinguish between two storage types: *shared storage* containing original items accessible to all users (not counted toward space complexity), and *per-user storage* for additional memory each user needs to compute personalized recommendations, including document identifiers and user-item association probabilities. Throughout this paper, "space complexity" refers exclusively to per-user storage.

Our contributions. We reduce *S3MOR* to the problem of *streaming submodular maximization under a partition matroid constraint*, which allows us to leverage existing online streaming algorithms. Based on this reduction, we propose three algorithms and evaluate them across multiple dimensions: competitive ratio, space complexity, time complexity, and response time complexity. We state our main results as follows:

- We prove that provided sufficient memory to store the whole stream, a greedy algorithm, LMGREEDY, can achieve the best possible $1/2$ competitive ratio for the *S3MOR* problem.
- If the exact number T of user visits is known, the reduction preserves the competitive ratio of the underlying online streaming submodular algorithm.
- However, in realistic scenarios, the number of user visits T is unknown and we need to guess the number of visits as $T' > T$. For this case, we introduce an algorithm STORM, which has a competitive ratio of $\frac{1}{4(T'-T+1)}$.
- To achieve a better competitive ratio, we propose a second algorithm STORM++ that makes multiple concurrent guesses of T, maintaining one solution per guess. It then greedily aggregates outcomes across guesses. This algorithm achieves a competitive ratio of $1/(8\delta)$,

where δ is a tunable parameter that balances efficiency and solution quality. The space and time complexity increase by a factor of $\delta T'$ relative to the underlying streaming algorithm.

- STORM and STORM++ achieve worse competitive ratios than LMGREEDY, but they utilize much smaller space and have significantly better response time.

We summarize the results for all algorithms mentioned above in Table 1. We validate our approach through empirical experiments on both large-scale and small-scale real-world datasets. Our results show the effectiveness of our algorithms in terms of coverage quality and memory usage, in comparison to relevant baselines.

Notation and background. We assume familiarity with constrained submodular maximization and competitive analysis. We provide formal definitions in Appendix A.

Related work

Submodular optimization plays a fundamental role in machine learning, combinatorial optimization, and data analysis [13, 26, 29], capturing problems with diminishing returns, and finding applications in data summarization [34, 42], non-parametric learning [17, 45], recommendation systems [1, 33, 38, 44], influence maximization [24], and network monitoring [13]. The classical greedy algorithm achieves a $(1 - 1/e)$-approximation for monotone submodular maximization under cardinality constraints [39], and extends to matroid constraints [6, 12].

We study stochastic submodular coverage functions, where the objective depends on random realizations but selections must be made non-adaptively. This contrasts with adaptive submodularity [16, 18, 37, 41], which models sequential decision-making with feedback and supports strong guarantees for greedy strategies. Related work includes stochastic variants of set and submodular cover, where the objective itself is random; notable examples analyze adaptivity gaps and approximation bounds under oracle access [15, 19].

Classical greedy algorithms for (constrained) submodular optimization [27, 35, 39] assume random access to the full dataset, which is infeasible in large-scale or streaming settings. This challenge has motivated streaming algorithms that make irrevocable decisions under memory constraints [4, 5, 9, 22, 23, 36]. Buchbinder et al. [5] apply the preemption paradigm, i.e., replacing elements with better ones, to achieve a tight $(1/2-\varepsilon)$-approximation for monotone functions under cardinality constraints. Sieve++ [23] uses thresholding to achieve a $(1/2 - \varepsilon)$-approximation under cardinality constraints using only $\mathcal{O}(k)$ memory. Extensions to matroid and partition constraints include [7, 8, 10]; for instance, Chekuri et al. [8] gives a $1/4p$-competitive online streaming algorithm for monotone p-matchoids. These approaches, however, do not handle stochastic item utilities or repeated, unpredictable user access. We address this gap by studying stochastic submodular coverage under partition constraints in a streaming model with limited memory and unknown user access requests.

To extend submodularity to sequences, Alaei et al. [1] introduce sequence-submodularity and sequence-monotonicity, showing that a greedy algorithm achieves a $(1 - 1/e)$-approximation, with applications in online ad allocation. Similarly, Ohsaka and Yoshida [40] propose k-submodular functions to model optimization over k disjoint subsets, providing constant-factor approximations for monotone cases. In contrast to their settings, our problem involves making irrevocable decisions as an incoming stream of items progresses, requiring each of the k subsets to be constructed incrementally rather than all at once after seeing the entire input.

Finally, diversity-aware recommendation and coverage problems are closely related, as many systems must recommend item sets rather than single items, motivating submodular approaches to maximize coverage and user satisfaction [2, 43]. Ashkan et al. [3] propose a greedy offline method for maximizing utility under a submodular diversity constraint. Yue and Guestrin [44] use submodular bandits for diversified retrieval, learning user interaction probabilities adaptively rather than relying on a known oracle, as in our setting.

2 Streaming submodular maximization with on-demand requests

Problem setting. Let $\mathcal{V} = \{V_1, V_2, \ldots, V_N\}$ be an ordered set of N news items appearing in the stream, where item V_i arrives at time step i. Each news item $V_i \in \mathcal{V}$ covers a subset of topics from a predefined set of topics $C = \{c_1, c_2, \ldots, c_d\}$, where d denotes the total number of topics.

Formally, $V_i \subseteq C$ for each $i \in [N]$. Additionally, each news item V_i is associated with a probability[1] $p_i \in [0, 1]$, modeling the probability that a user will click the item when presented to them. When a user clicks a news item, we say that the user is exposed to all topics in the news item.

Moreover, we assume that we observe a binary variable $\tau_i \in \{0, 1\}$ indicating whether the user accesses the system at time step i. Specifically, $\tau_i = 1$ denotes an access at time step i, and $\tau_i = 0$ otherwise. Whenever $\tau_i = 1$, the system should present at most k news items from the available news items $\{V_1, \ldots, V_i\}$. Now, suppose a user visits the system T times. The system outputs T sets $\mathcal{S}^1, \cdots, \mathcal{S}^T$, where $\mathcal{S}^t$ represents the items presented to the user at their t-th visit, for $t \in [T]$. We allow the same item to be presented at multiple visits; however, we treat each appearance as a distinct copy. We make this choice as the user has multiple chances to click on an item if the item is presented multiple times. Therefore, for any $t \neq s$, we have $\mathcal{S}^t \cap \mathcal{S}^s = \emptyset$. We define the set of all items presented to the user over the entire sequence of visits as $\mathcal{S} = \bigcup_{t=1}^{T} \mathcal{S}^t$.

Objective function. We define $f(\mathcal{S})$ as the expected number of topics the user covers after T visits, where the expectation is taken over click probabilities. Let $\sigma_j \in \{0, 1\}$ indicate whether topic c_j is covered, i.e., the user clicked at least one item that covers c_j from the shown sets $\mathcal{S}^1, \cdots, \mathcal{S}^T$. The probability $\Pr(\sigma_j = 0)$ that topic c_j remains uncovered equals the probability the user clicks no items covering c_j.

Since the user clicks on item V_i with probability p_i, the probability that they do not click on it is $1 - p_i$ and, thus, $\Pr(\sigma_j = 0) = \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i)$, leading to the expectation $\mathbb{E}[\sigma_j] = 1 - \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i)$. By linearity of expectation, the expected number of topics covered by a user, which we denote by $f(\mathcal{S})$, is given by

$$f(\mathcal{S}) = \mathbb{E}\left[\sum_{j=1}^{d} \sigma_j\right] = \sum_{j=1}^{d} \mathbb{E}[\sigma_j] = \sum_{j=1}^{d} \left(1 - \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i)\right). \tag{1}$$

Lemma 2.1. *$f(\mathcal{S})$ is a non-decreasing submodular set function.*

Problem definition. We formally define our new problem *streaming stochastic submodular maximization with on-demand requests* (*S3MOR*) below.

Problem 2.2 (***S3MOR***). *We define a news item stream $\mathbf{S}$ as a sequence of triples: $\mathbf{S} = ((V_1, p_1, \tau_1), \ldots, (V_N, p_N, \tau_N))$ where N is the (unknown) length of the stream. At each time step i, the system receives a triple (V_i, p_i, τ_i), where $V_i \in \mathcal{V}$, $p_i \in [0, 1]$, and $\tau_i \in \{0, 1\}$. Whenever $\tau_i = 1$, the system selects up to k news items from the set of items $\{V_1, \ldots, V_i\}$ received so far to present to the user. Let $\mathcal{S} = \{\mathcal{S}^1, \ldots, \mathcal{S}^T\}$ denote the sets of news items presented to the user over T accesses. The objective is to maximize the expected number of distinct topics the user is exposed to by the end of the stream, measured by the function $f(\mathcal{S})$.*

There are two key challenges in our streaming setting: (i) the system does not know in advance when or how many times a user will access it; and (ii) the system has to make decisions based on the information available so far, and these decisions are irrevocable. In the following, we show how the problem can be reduced to a submodular maximization problem subject to a partition matroid constraint. We then leverage this connection to develop solutions under various memory constraints.

Reduction to submodular maximization under a partition matroid constraint. We first reduce the *S3MOR* problem (Problem 2.2) to submodular maximization under a partition matroid constraint, which enables us to apply efficient streaming algorithms with provable guarantees. Let $r \colon t \mapsto r_t$ map a user's t-th access to its access time, and $(r_t)_{t=1}^{T}$ be the sequence of times at which the user accesses the system over an unknown time period. Whenever the user accesses the system, it presents up to k news items selected from all the news items available since the beginning of the stream, denoted by $\{V_i\}_{i=1}^{r_t}$.

[1]These probabilities can be interpreted as measures of user-item preference or engagement, and are typically estimated from historical click-through data using machine learning models such as logistic regression, factorization machines, or neural networks trained on user behavior patterns and item features. We assume they are static and remain constant over time.

Let V_i^t be a *copy* of the item V_i associated with the t-th visit. Let $\mathcal{V}^t$ be a collection of these copies until the t-th visit, namely, $\mathcal{V}^t = \{V_i^t\}_{i=1}^{r_t} = \{V_1^t, \ldots, V_{r_t}^t\}$. Without loss of generality, we treat each copy as a distinct item, i.e., $V_i^t \neq V_i^s$ for all $t \neq s$. Clearly, $\mathcal{V}^t \cap \mathcal{V}^s = \emptyset$ for $t \neq s$.

Moreover, let $\mathcal{S}^t \subseteq \mathcal{V}^t$ be the set of news items that the system presents to the user at their t-th visit. Let $\mathcal{S} = \bigcup_{q=1}^{T} \mathcal{S}^q$ be the union of all sets of presented items. Analogously to Equation (1), we adapt the function $f\colon 2^{\cup_{t=1}^{T}\mathcal{V}^t} \to \mathbb{R}$ such that $f(\mathcal{S})$ is the expected number of topics covered by the (copies) of news items that the user clicked on after $\mathcal{S}^1, \ldots, \mathcal{S}^T$ have been presented to the user.

Observation 2.3. *Let $(\cup_{t=1}^{T}\mathcal{V}^t, \mathcal{F})$ be a set system, where $\mathcal{F}$ is a collection of subsets of $\cup_{t=1}^{T}\mathcal{V}^t$, and $\mathcal{F}$ is constructed in the following way: for any set $X \subseteq \cup_{t=1}^{T}\mathcal{V}^t$, if $|X \cap \mathcal{V}^t| \leq k$ for any $1 \leq t \leq T$, then $X \in \mathcal{F}$. Observe that $(\cup_{t=1}^{T}\mathcal{V}^t, \mathcal{F})$ is a partition matroid. Problem 2.2 can be equivalently formulated as $\max_{X \in \mathcal{F}} f(X)$.*

Based on Observation 2.3, we develop a baseline algorithm for the Problem 2.2. Suppose that the system stores all the news items up to when the user accesses the system at the t-th time, and also has stored all the previous news item sets presented to the user, i.e., $\cup_{q=1}^{t-1}\mathcal{S}^q$. In this case, the stored news items up to time r_t constitute one partition of the matroid, i.e., $\mathcal{V}^t = \{V_i^t\}_{i=1}^{r_t}$. We then apply the local greedy algorithm by Fisher et al. [12], i.e., the system greedily selects k news items from $\mathcal{V}^t$ that maximize the marginal gain of the objective function with respect to $f(\cup_{q=1}^{t-1}\mathcal{S}^q)$. This algorithm does not require any knowledge of T, and has a competitive ratio of $1/2$. Since all incoming news items need to be stored, the memory usage is $\Omega(N + kT)$, and the running time is $\sum_{q=1}^{T} \mathcal{O}(|\mathcal{V}^q|k)$. The response time is $\mathcal{O}(Nk)$ since it selects k items iteratively from $\mathcal{O}(N)$ items. We present the algorithm in Algorithm 3 (LMGREEDY) in Appendix B.

Theorem 2.4. *Algorithm 3 for Problem 2.2 has a competitive ratio of $1/2$, space complexity $\Theta(N + kT)$, time complexity $\mathcal{O}(NTk)$, and response time complexity $\mathcal{O}(Nk)$. Moreover, the competitive ratio of $1/2$ is tight, i.e., for Problem 2.2, no streaming algorithm can achieve a competitive ratio better than $1/2$ without violating the irrevocability constraint.*

The drawback of LMGREEDY is that its memory usage depends linearly on the stream length, making it infeasible for large streams. Thus, we consider the setting where the system has limited memory and can store only a user-specific subset of news items.

3 Algorithms with limited memory

We now consider a more realistic setting where the system has limited memory. Our objective is to substantially reduce the memory usage and the response time complexity while achieving a bounded approximation ratio. We first establish that any algorithm achieving reasonably good performance must use at least $\Theta(Tk)$ memory.

To illustrate this lower bound, consider the following example: Suppose that a user makes T requests; the input stream contains more than Tk items, each containing one distinct topic and associated with acceptance probability of 1. At the end of the stream, the user sequentially submits T requests. If an algorithm is allocated with only $\Theta(T'k)$ memory for some $T' \leq T$, it can store at most $\Theta(T'k)$ items, thus, at most $\Theta(T'k)$ distinct topics. However, optimally, the user may access $\Theta(Tk)$ distinct topics. We can conclude that for this example, the competitive ratio is at most $\Theta(T'/T)$, which is arbitrarily poor when $T' \ll T$.

Since the exact number of requests T is typically unknown in advance, we assume the system has access to an upper bound $T' \geq T$ and is allocated $\Omega(kT')$ memory. This allocation is sufficient to store all candidate items throughout the user's interaction period. In practice, T' can be estimated from historical interaction data, allowing the system to set T' close to T within reasonable accuracy, ensuring both efficient memory utilization and meaningful performance guarantees.

3.1 A first memory-efficient streaming algorithm

Our first memory-efficient algorithm processes the incoming stream $\mathbf{S}$ as follows. First, it initializes T' empty candidate sets $\mathcal{A}^i$ for $i \in [T']$, all of which are initially *active*. Upon the arrival of a news item from the stream $\mathbf{S}$, for each active candidate set, the algorithm either adds the item to the set

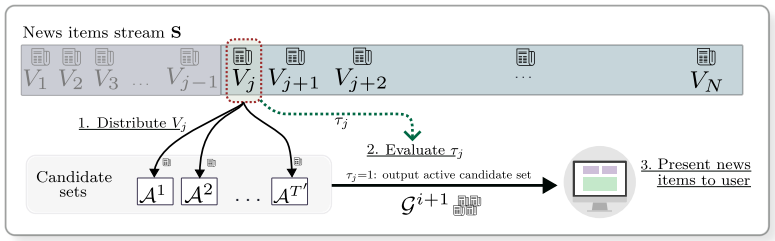


Figure 1: Schematic view of our algorithm STORM. We first initialize T' empty active candidate sets. For each incoming news item in the stream $\mathbf{S}$, we decide whether it can be added to/swapped into each of the active candidate sets. When a user submits a request, i.e., $\tau_j = 1$, we select the best active candidate set, present it to the user, and deactivate it.

Algorithm 1: STORM

Input: Estimated visits $T' \in \mathbb{N}$, budget $k \in \mathbb{N}$.

$\mathcal{A}^1, \cdots, \mathcal{A}^{T'} \leftarrow \emptyset, \mathcal{T} = \{1, \cdots, T'\}, \mathcal{G}^0 \leftarrow \emptyset, i \leftarrow 0$
for (V, τ, p) *in the stream* **do**
  for $j \in \mathcal{T}$ **do**
    if $|\mathcal{A}^j| < k$ **then**
      $\mathcal{A}^j \leftarrow \mathcal{A}^j \cup \{V\}$
    else
      $V' \leftarrow \arg\min_{V \in \mathcal{A}^j} \nu(f, \cup_{q \in [T']} \mathcal{A}^q, V)$
      if $f(V \mid \cup_{q \in [T']} \mathcal{A}^q) \geq 2\nu(f, \cup_{q \in [T']} \mathcal{A}^q, V')$ **then**
        $\mathcal{A}^j \leftarrow \mathcal{A}^j \setminus \{V'\} \cup \{V\}$
  if $\tau = 1$ **then**
    $j^* \leftarrow \arg\max_{j \in \mathcal{T}'} f(\mathcal{A}^j | \bigcup_{t=0}^{i} \mathcal{G}^t)$
    $\mathcal{T} \leftarrow \mathcal{T} \setminus \{j^*\}$ ▷ Deactivate candidate set $\mathcal{A}^{j*}$
    $\mathcal{G}^{i+1} \leftarrow \mathcal{A}^{j^*}$
    output $\mathcal{G}^{i+1}$
    $i \leftarrow i + 1$

or replaces an existing item if the set is full. When a user accesses the system, the algorithm selects one of the active candidate sets to present to the user. The chosen candidate set is then marked as *inactivate* and will not be updated further. Figure 1 shows an overview.

Based on our reduction to submodular maximization under a partition matroid constraint, we can adapt any streaming algorithm designed for matroid constraints, provided it produces irrevocable outputs. Notable examples include the algorithms proposed by Chakrabarti and Kale [7], Chekuri et al. [8], and Feldman et al. [10], all of which achieve a $1/4$ approximation ratio. In our work, we specifically adapt the algorithm introduced by Chekuri et al. [8].

Our algorithm, named STORM, is shown in Algorithm 1. It keeps track of the number of visits using a counter i. Assume the user has visited i times when news item V_j arrives. This implies that, prior to the arrival of V_j, i active sets from i partitions have already been presented to the user, and thus, neither V_j nor any subsequent item will be added to these partitions. The algorithm iterates over the remaining $T' - i$ active candidate sets, and determines whether V_j should be added to any of them, with replacement if necessary.

The decision to replace an existing item V with the current item V_j is based on the comparison of their incremental value with respect to set $\cup_{t \in [T']} \mathcal{A}^t$. Specifically, we adopt the notation $\nu(f, \cup_{t \in [T']} \mathcal{A}^t, V)$ from Chekuri et al. [8] and define

$$\nu(f, \cup_{t \in [T']} \mathcal{A}^t, V) = f_{\widehat{\cup_{t \in [T']} \mathcal{A}^t}}(V), \text{ with } \widehat{\cup_{t \in [T']} \mathcal{A}^t} = \{V' \in \cup_{t \in [T']} \mathcal{A}^t \colon V' < V\}.$$

Algorithm 2: STORM++

Input: Estimated visits $T' \in \mathbb{N}$, budget $k \in \mathbb{N}$, parameter $\delta \in \mathbb{N}$.

1 $\mathcal{G}^0 \leftarrow \emptyset$, $\mathcal{B}^0 \leftarrow \emptyset$, $i \leftarrow 0$, $\mathcal{P} \leftarrow \left\{\delta, 2\delta, \ldots, \lceil \frac{T'}{\delta} \rceil \delta\right\}$

2 **for** $\tilde{T} \in \mathcal{P}$ **do** Initialize STORM $(\tilde{T}, k)$ ▷ Maintain STORM for different $\tilde{T}$

3 **for** (V, τ, p) *in the stream* **do**

4 **for** $\tilde{T} \in \mathcal{P}$ **do**

5   $\mathcal{G}^{i+1,(\tilde{T})} \leftarrow$ STORM $(\tilde{T}, k)$.`step`(V, τ, p) ▷ Process (V, τ, p) by executing lines 3-15 of Alg 1

6 **if** $\tau = 1$ **then**

7   $\tilde{T}^* \leftarrow \arg\max_{\tilde{T} \in \mathcal{P}} f(\mathcal{G}^{i+1,(\tilde{T})} \mid \bigcup_{j=0}^{i} \mathcal{B}^j)$

8   $\mathcal{B}^{i+1} \leftarrow \mathcal{G}^{i+1,(\tilde{T}^*)}$

9   **output** $\mathcal{B}^{i+1}$

10   $i \leftarrow i + 1$

Here, $V' < V$ indicates item V' is added to the set $\cup_{t \in [T']} \mathcal{A}^t$ before item V. For a specific set $\mathcal{A}^{t'}$, V_j replaces an item in $\mathcal{A}^{t'}$ if $f_{\cup_{t \in [T']} \mathcal{A}^t}(V_j) \geq 2 \cdot \min_{V \in \mathcal{A}^{t'}} \nu(f, \cup_{t \in [T']} \mathcal{A}^t, V)$.

Upon the t-th user visit, the system selects which active candidate set to output in a greedy manner. To be specific, let $\mathcal{G}^1, \cdots, \mathcal{G}^{t-1}$ denote the candidate sets that have already been presented to the user, and $\mathcal{T}$ represent the indices of the remaining active sets. The system selects the active set that offers the highest incremental gain with respect to the union of previously presented sets. Formally, let $j^* = \arg\max_{j \in \mathcal{T}} f(\mathcal{A}^j | \cup_{j=1}^{t-1} \mathcal{G}^j)$, and the system outputs $\mathcal{G}^t = \mathcal{A}^{j^*}$.

Theorem 3.1. *Let T be the number of user accesses and T' a given upper bound. Algorithm 1 has competitive ratio $\frac{1}{4(T'-T+1)}$, space complexity $\mathcal{O}(T'k)$, time complexity $\mathcal{O}(NkT')$, and response time complexity $\mathcal{O}(T')$.*

If the exact number of visits T is known, we can set the upper bound $T' = T$ and obtain a $1/4$-competitive ratio using Algorithm 1. However, the competitive ratio of Algorithm 1 can be arbitrarily bad if T' is significantly larger than T. Note that our analysis is tight. We can demonstrate that for any fixed T, there exists an example such that the competitive ratio of Algorithm 1 is at most $\frac{1}{T'-T+1}$, which is only a constant factor away from our analysis. We provide the proof in Appendix C.

3.2 A memory-efficient streaming algorithm with bounded competitive ratio

As shown in the previous section, to achieve a large enough competitive ratio with STORM, the system must have a good enough estimate of the number of user visits T. However, in practice, this information is typically unavailable in advance. To address this issue, we discretize the range $[T']$ to generate multiple guesses for the value of T, and run a separate instance of Algorithm 1 for each guess. When a user visits the system, each copy of Algorithm 1 outputs a solution, and we select the best solution over all the outputs.

Specifically, Algorithm 2 (named STORM++) first initializes the set of guesses of T as $\mathcal{P} = \{\delta i \mid i \in [\lceil T'/\delta \rceil]\}$, where $\delta \in \mathbb{N}$ is a parameter controlling the trade-off between competitive ratio as well as the space and time complexity. In this construction, there always exists a guess $\tilde{T} \in \mathcal{P}$ that is between T and $T + \delta$. Therefore, if we would run Algorithm 1 with $\tilde{T}$ as the input parameter, and present the results to the user, we are guaranteed to have a competitive ratio of at least $1/4\delta$ as shown in Theorem 3.1; however, we do not know the value of $\tilde{T}$ in advance. Thus, we adopt a greedy strategy to aggregate the results from all the copies of Algorithm 1 to determine an output for each user visit. At the i-th user visit, we collect the i-th output from each running copy of STORM with a different guess $\tilde{T} \in \mathcal{P}$ and select the one that maximizes the incremental gain. This selection rule leads to the following result.

Theorem 3.2. *Algorithm 2 has a competitive ratio of $1/(8\delta)$, space complexity $\mathcal{O}(T'^2 k/\delta)$, time complexity $\mathcal{O}(NkT'^2/\delta)$, and response time complexity $\mathcal{O}(T')$.*

4 Empirical evaluation

We empirically evaluate the performance of our proposed algorithms STORM and STORM++. We denote STORM(T) and STORM(T') as the STORM algorithm with the input parameter set to T (i.e., exact number of user accesses) and T' (i.e., upper bound of user accesses), resp.

We design our experiments to answer the following research questions: **(Q1)** How effective are the solutions provided by STORM++ and STORM(T') compared to other baselines and to each other? **(Q2)** What is the impact of parameters δ and k on the performance of STORM++ and STORM(T')? **(Q3)** How sensitive are STORM++ and STORM(T') to the upper bound T' and do our algorithms require highly-accurate estimation of T' to perform well?

We use the following **baselines**:

- LMGREEDY: Our upper-bound baseline, which assumes linear memory size in the stream length, i.e., potentially unbounded memory (Algorithm 3).
- SIEVE++: Based on Kazemi et al. [23], we implement a heuristic version of the SIEVE++ algorithm to solve the *S3MOR* problem. The adaptation proceeds as follows: (1) Before the first user visit, we run the exact SIEVE++ algorithm and obtain output set $\mathcal{S}^1$. (2) For each subsequent i-th output, we run SIEVE++ on the data stream between the $(i-1)$-th and i-th user visits, using the marginal gain function $f(\cdot|\bigcup_{j=1}^{i-1}\mathcal{S}^i)$ as the objective.
- PREEMPTION: We adapt the algorithm proposed by Buchbinder et al. [5] for solving the *S3MOR* problem in the same way as described above for SIEVE++.

Datasets: We use six real-world datasets, including four rating datasets KuaiRec, Anime, Beer and Yahoo, and two multi-label dataset RCV1 and Amazon. The number of items these dataset contains vary from thousands to one million, and the topic sets size vary from around 40 to around 4000. We provide the details in Appendix D.1.

Experimental setting: To simulate the news item stream, we randomly shuffle the items to form a stream of length N. To simulate the user visit sequence, we start with an all-zeros sequence and then randomly select T indices, and set those positions to one. We establish an upper bound on the number of visits by setting $T' = T + \Delta T$ for some positive ΔT. We generate synthetic click probabilities, and provide the details in Appendix D.1.

In all experiments, we randomly selected 50 users and report the average expected coverage for the Yahoo, RCV1, and Amazon datasets. Standard deviations and expected coverage results for the other three datasets are provided in Appendix D. We also report runtime and memory usage on the largest dataset, Amazon.

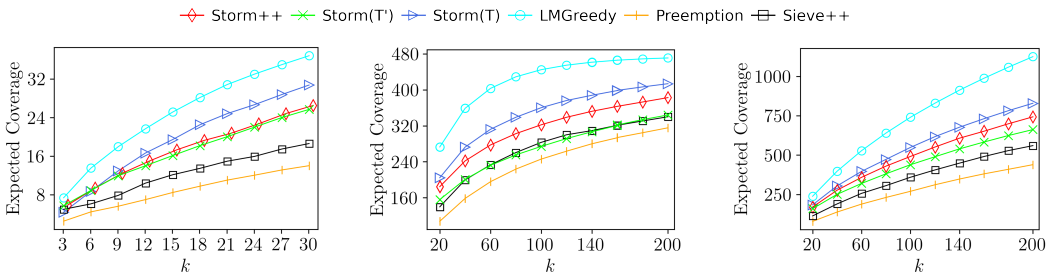
All experiments ran on a MacBook Air (Model Mac15,12) equipped with an Apple M3 chip (8-core: 4 performance and 4 efficiency cores), 16 GB of memory, and a solid-state drive (SSD). The source code and datasets are available.[2]

Results and discussion

Impact of k and T on the expected coverage. As shown in Figure 2, LMGREEDY achieves the best performance across all datasets and values of k. Among our proposed methods, STORM(T) consistently performs the best, followed by STORM++, then STORM(T'). Compared to baselines SIEVE++ and PREEMPTION, our algorithms generally perform better across datasets and values of k, with exceptions on RCV1, where PREEMPTION performs comparably to STORM(T'). In Figure 3, we observe a trend shift: STORM++ slightly outperforms STORM(T') and STORM(T) across datasets and T values. STORM(T') and STORM(T) perform comparably. This performance shift is due to smaller δ and ΔT in this experiment, which lead to improved competitive ratios for STORM(T') and STORM++. Although STORM(T) offers the strongest theoretical guarantee of the three algorithms, STORM(T') and STORM++ can empirically outperform it (we give an example of this effect in Appendix D.2).

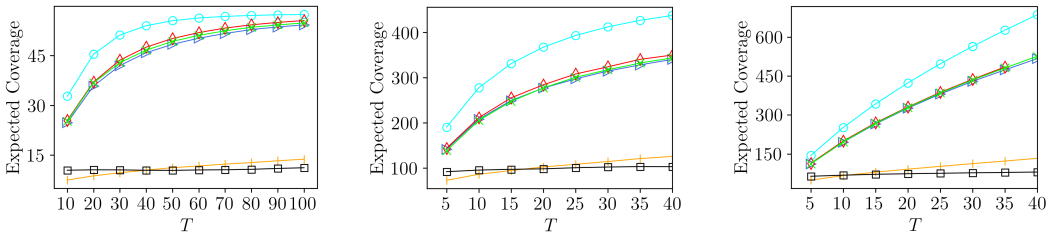
Performance regarding parameters δ and ΔT. Figure 4a shows that STORM++'s expected coverage decreases as δ increases, consistent with theory: as δ grows from 3 to 30, the performance guar-

[2] https://github.com/HongLWang/Streaming-Submodular-maximization-with-on-demand-user-request



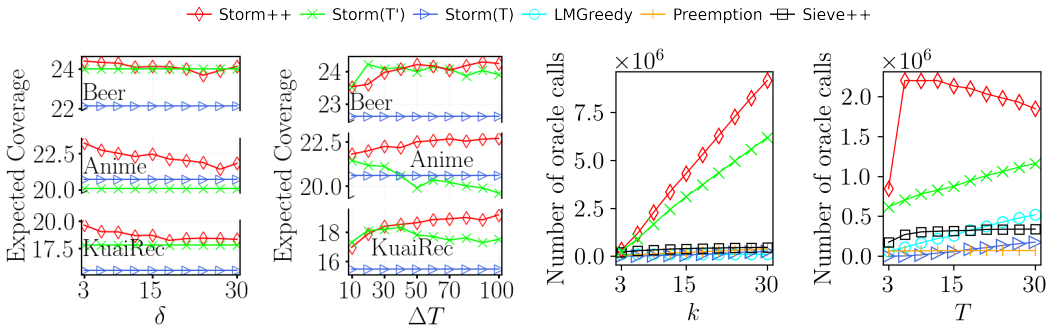
(a) Yahoo (b) RCV1 (c) Amazon

Figure 2: Empirical variation in expected coverage as a function of budget k on all datasets. Parameter $T = 5$, $\delta = 25$ and $\Delta T = 45$ are fixed



(a) Yahoo (b) RCV1 (c) Amazon

Figure 3: Empirical variation in expected coverage as a function of number of visits T on all datasets. Parameter $k = 10$, $\delta = 10$ and $\Delta T = 10$ are fixed.



(a) (b) (c) (d)

Figure 4: Left two: Impact of ΔT and δ on expected coverage. In (a) and (b), we fix $k = 5$ and $T = 10$. When varying δ, we fix $\Delta T = 60$; when varying ΔT, we fix $\delta = 10$. Right two: Oracle call counts on the Beer dataset. We fix $\delta = 25$ and $\Delta T = 45$. When varying k, we fix $T = 5$; when varying T, we fix $k = 5$.

Table 2: Runtime and mem. usage for the Amazon dataset. We set k, T, and δ to 10, $T' = 20$.

	STORM++	STORM(T')	STORM(T)	LMGREEDY	SIEVE++	PREEMPTION
Runtime (s)	35.13	7.94	4.38	88.13	307.43	136.04
Memory (MB)	2.00	1.28	0.72	28.99	0.91	0.03

antee drops by a factor of 10. This is reflected empirically with coverage declines across datasets, for example, from 130 to 120 on Amazon.

Figure 4b shows that STORM(T')'s performance declines with increasing ΔT, matching the theoretical drop in competitive ratio (e.g., 10-fold decrease from $\Delta T = 10$ to 100). In contrast, STORM++ is theoretically unaffected.

Number of oracle calls, runtime and memory. Figure 4c shows that the number of oracle calls increases rapidly and linearly with budget k for both STORM(T') and STORM++, while remaining

relatively stable for other algorithms. Similarly, Figure 4d indicates a linear increase in oracle calls with user visits T for STORM(T'), STORM(T), and LMGREEDY, but a much slower growth for SIEVE++ and PREEMPTION. Notably, Figure 4d also reveals a non-monotonic trend: oracle calls first rise, then decline with increasing T. This increase is due to the guessing strategy for T under $\delta = 25$ and $\Delta T' = 45$, where the number of guesses $\mathcal{T}$ increases ($\mathcal{T} = \{25, 50\}$ for $T = 3$ and $\mathcal{T} = \{25, 50, 75\}$ for $T \in [6, 30]$). The decrease is due to STORM++'s irrevocability—as more subsets are presented to the user, fewer items remain in $\mathcal{S}$ for potential swap, thereby reducing the number of subsequent oracle calls.

Table 2 shows that our proposed algorithms require much less memory than LMGREEDY, and use comparable memory to the other two baselines. STORM(T') and STORM(T) are also significantly faster than the remaining algorithms.

5 Conclusion

We investigate a novel online setting for streaming submodular maximization where users submit on-demand requests throughout the stream. In this context, we introduce the *Streaming Stochastic Submodular Maximization with On-demand Requests* (*S3MOR*) problem with the goal of maximizing the expected coverage of the selected items. We first propose a memory-efficient approximation algorithm that highlights the trade-off between memory usage and solution quality, but its competitive ratio can be poor. To solve this, we propose a parameter-dependent memory-efficient approximation algorithm to trade off between the competitive ratio and the memory and time usage. Our empirical evaluations show that our proposed algorithms consistently outperform the baselines. We believe our approach will have a positive social impact. For instance, it can be applied to enhance the diversity of news content recommended to the users, thereby helping to expose them to multiple viewpoints and broaden their knowledge. While our approach optimizes topic coverage and can be used to promote content diversity, its misuse, such as deliberately optimizing a submodular utility function that favors polarized or biased content, could risk amplifying existing biases. However, such outcomes would require intentional manipulation of the objective function. When applied responsibly, our approach does not pose such risks.

Acknowledgments and Disclosure of Funding

This research is supported by the ERC Advanced Grant REBOUND (834862), the Swedish Research Council project ExCLUS (2024-05603), and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

[1] Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. 2021. Maximizing sequence-submodular functions and its application to online advertising. *Management Science* 67, 10 (2021), 6030–6054.

[2] Bushra Alhijawi, Arafat Awajan, and Salam Fraihat. 2022. Survey on the objectives of recommender systems: Measures, solutions, evaluation methodology, and new perspectives. *Comput. Surveys* 55, 5 (2022), 1–38.

[3] Azin Ashkan, Branislav Kveton, Shlomo Berkovsky, and Zheng Wen. 2015. Optimal Greedy Diversity for Recommendation.. In *IJCAI*, Vol. 15. 1742–1748.

[4] Ashwinkumar Badanidiyuru, Baharan Mirzasoleiman, Amin Karbasi, and Andreas Krause. 2014. Streaming submodular maximization: Massive data summarization on the fly. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 671–680.

[5] Niv Buchbinder, Moran Feldman, and Roy Schwartz. 2019. Online submodular maximization with preemption. *ACM Transactions on Algorithms (TALG)* 15, 3 (2019), 1–31.

[6] Gruia Calinescu, Chandra Chekuri, Martin Pal, and Jan Vondrák. 2011. Maximizing a monotone submodular function subject to a matroid constraint. *SIAM J. Comput.* 40, 6 (2011), 1740–1766.

[7] Amit Chakrabarti and Sagar Kale. 2015. Submodular maximization meets streaming: matchings, matroids, and more. *Mathematical Programming* 154 (2015), 225–247.

[8] Chandra Chekuri, Shalmoli Gupta, and Kent Quanrud. 2015. Streaming algorithms for submodular function maximization. In *Automata, Languages, and Programming: 42nd International Colloquium, ICALP 2015, Kyoto, Japan, July 6-10, 2015, Proceedings, Part I 42*. Springer, 318–330.

[9] Marwa El Halabi, Federico Fusco, Ashkan Norouzi-Fard, Jakab Tardos, and Jakub Tarnawski. 2023. Fairness in streaming submodular maximization over a matroid constraint. In *International Conference on Machine Learning*. PMLR, 9150–9171.

[10] Moran Feldman, Amin Karbasi, and Ehsan Kazemi. 2018. Do less, get more: Streaming submodular maximization with subsampling. *Advances in Neural Information Processing Systems* 31 (2018).

[11] Moran Feldman, Paul Liu, Ashkan Norouzi-Fard, Ola Svensson, and Rico Zenklusen. 2021. Streaming submodular maximization under matroid constraints. *arXiv preprint arXiv:2107.07183* (2021).

[12] Marshall L Fisher, George L Nemhauser, and Laurence A Wolsey. 1978. *An analysis of approximations for maximizing submodular set functionsII*. Springer.

[13] Satoru Fujishige. 2005. *Submodular functions and optimization*. Vol. 58. Elsevier.

[14] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 540–550.

[15] Michel Goemans and Jan Vondrák. 2006. Stochastic covering and adaptivity. In *Latin American symposium on theoretical informatics*. Springer, 532–543.

[16] Daniel Golovin and Andreas Krause. 2011. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *Journal of Artificial Intelligence Research* 42 (2011), 427–486.

[17] Ryan Gomes and Andreas Krause. 2010. Budgeted Nonparametric Learning from Data Streams.. In *ICML*, Vol. 1. Citeseer, 3.

[18] Alkis Gotovos, Amin Karbasi, and Andreas Krause. 2015. Non-Monotone Adaptive Submodular Maximization. In *IJCAI*. 1996–2003.

[19] Fabrizio Grandoni, Anupam Gupta, Stefano Leonardi, Pauli Miettinen, Piotr Sankowski, and Mohit Singh. 2008. Set covering with our eyes closed. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. IEEE, 347–356.

[20] Ran Haba, Ehsan Kazemi, Moran Feldman, and Amin Karbasi. 2020. Streaming submodular maximization under a k-set system constraint. In *International Conference on Machine Learning*. PMLR, 3939–3949.

[21] Chien-Chung Huang, Naonori Kakimura, and Yuichi Yoshida. 2020. Streaming algorithms for maximizing monotone submodular functions under a knapsack constraint. *Algorithmica* 82, 4 (2020), 1006–1032.

[22] Chien-Chung Huang, Theophile Thiery, and Justin Ward. 2021. Improved multi-pass streaming algorithms for submodular maximization with matroid constraints. *arXiv preprint arXiv:2102.09679* (2021).

[23] Ehsan Kazemi, Marko Mitrovic, Morteza Zadimoghaddam, Silvio Lattanzi, and Amin Karbasi. 2019. Submodular streaming in all its glory: Tight approximation, minimum memory and low adaptive complexity. In *International Conference on Machine Learning*. PMLR, 3311–3320.

[24] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.

[25] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

[26] Andreas Krause and Daniel Golovin. 2014. Submodular function maximization. *Tractability* 3, 71-104 (2014), 3.

[27] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. 2007. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 420–429.

[28] David D Lewis, Yiming Yang, Tony G Rose, and Fan Li. 2004. Rcv1: A new benchmark collection for text categorization research. *Journal of machine learning research* 5, Apr (2004), 361–397.

[29] Yajing Liu, Edwin KP Chong, Ali Pezeshki, and Zhenliang Zhang. 2020. Submodular optimization problems and greedy strategies: A survey. *Discrete Event Dynamic Systems* 30, 3 (2020), 381–412.

[30] Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: understanding rating dimensions with review text. In *Proceedings of the 7th ACM conference on Recommender systems*. 165–172.

[31] Julian McAuley, Jure Leskovec, and Dan Jurafsky. 2012. Learning attitudes and attributes from multi-aspect reviews. In *2012 IEEE 12th International Conference on Data Mining*. IEEE, 1020–1025.

[32] Julian John McAuley and Jure Leskovec. 2013. From amateurs to connoisseurs: modeling the evolution of user expertise through online reviews. In *Proceedings of the 22nd international conference on World Wide Web*. 897–908.

[33] Anay Mehrotra and Nisheeth K Vishnoi. 2023. Maximizing submodular functions for recommendation in the presence of biases. In *Proceedings of the ACM Web Conference 2023*. 3625–3636.

[34] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, and Amin Karbasi. 2016. Fast constrained submodular maximization: Personalized data summarization. In *International Conference on Machine Learning*. PMLR, 1358–1367.

[35] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, Amin Karbasi, Jan Vondrák, and Andreas Krause. 2015. Lazier than lazy greedy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29.

[36] Baharan Mirzasoleiman, Stefanie Jegelka, and Andreas Krause. 2018. Streaming non-monotone submodular maximization: Personalized video summarization on the fly. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.

[37] Marko Mitrovic, Ehsan Kazemi, Moran Feldman, Andreas Krause, and Amin Karbasi. 2019. Adaptive sequence submodularity. *Advances in Neural Information Processing Systems* 32 (2019).

[38] Houssam Nassif, Kemal Oral Cansizlar, Mitchell Goodman, and SVN Vishwanathan. 2018. Diversifying music recommendations. *arXiv preprint arXiv:1810.01482* (2018).

[39] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functionsI. *Mathematical programming* 14 (1978), 265–294.

[40] Naoto Ohsaka and Yuichi Yoshida. 2015. Monotone k-submodular function maximization with size constraints. *Advances in Neural Information Processing Systems* 28 (2015).

[41] Shaojie Tang and Jing Yuan. 2020. Adaptive cascade submodular maximization. *arXiv preprint arXiv:2007.03592* (2020).

[42] Kai Wei, Rishabh Iyer, and Jeff Bilmes. 2015. Submodularity in data subset selection and active learning. In *International conference on machine learning*. PMLR, 1954–1963.

[43] Qiong Wu, Yong Liu, Chunyan Miao, Yin Zhao, Lu Guan, and Haihong Tang. 2019. Recent advances in diversified recommendation. *arXiv preprint arXiv:1905.06589* (2019).

[44] Yisong Yue and Carlos Guestrin. 2011. Linear submodular bandits and their application to diversified retrieval. *Advances in Neural Information Processing Systems* 24 (2011).

[45] Ghahramani Zoubin. 2013. Scaling the indian buffet process via submodular maximization. In *International Conference on Machine Learning*. PMLR, 1013–1021.